

SWE-Tester: Training Open-Source LLMs for Issue Reproduction in Real-World Repositories

Aditya Bharat Soni¹*, Rajat Ghosh², Vaishnavi Bhargava², Valerie Chen³, Debojyoti Dutta²

¹Carnegie Mellon University, ²Nutanix, ³Stanford University

Correspondence: adityabs@cs.cmu.edu, rajat.ghosh@nutanix.com, vaishnavi.bhargava@nutanix.com

Abstract

Software testing is essential for maintaining software reliability, yet issue reproduction tests are often missing when bugs are reported. Automatically generating such tests can accelerate debugging, support test-driven development, and improve automated software engineering systems that rely on execution-based verification. Existing approaches largely depend on proprietary language models, limiting accessibility and reproducibility. We present **SWE-Tester**¹ a framework for training open-source large language models to generate issue reproduction tests from natural language issue reports. To support this task, we curate a dataset of **41K** training instances from **2.6K** open-source GitHub repositories. Fine-tuning on this dataset improves issue reproduction performance across multiple model families, yielding gains of up to **10%** points in success rate and **21%** in change coverage on SWT-Bench Verified. We further study the effects of model size, training data scale, and inference-time compute, observing consistent improvements along all three axes. Our results demonstrate that open-source models can be effectively specialized for issue reproduction test generation and provide a scalable foundation for future software testing and automated issue-resolution systems.

1 Introduction

Large language models have demonstrated remarkable improvements on software engineering (SWE) tasks, as evidenced by progress on benchmarks like SWE-Bench (Jimenez et al., 2024), which evaluates an LLM’s ability to resolve codebase issues. However, issue resolution is only one component of the software development lifecycle. Rigorous software testing is equally critical, yet developers often find writing reproduction tests tedious (Straubinger and Fraser, 2023), and such tests are rarely present

when a bug is reported (Kang et al., 2023; Mündler et al., 2024).

Automating the generation of bug-reproducing tests can significantly enhance developer productivity, enable test-driven development (TDD) (Beck, 2022), and boost automated bug-fixing pipelines by serving as localized execution-based verifiers (Xia et al., 2024; Jain et al., 2025; Wei et al., 2025). Despite these benefits, existing state-of-the-art bug reproduction techniques, such as AssertFlip (Khatib et al., 2025) and e-Otter++, rely heavily on expensive, closed-source APIs (e.g., GPT-4o, Claude-3.7-Sonnet). In industrial environments, sending proprietary code to third-party APIs poses severe intellectual property and data privacy risks. Consequently, training local, open-source models for repository-level test generation is a critical operational necessity, yet it remains largely unexplored.

Furthermore, repository-level test reproduction is distinct from source-code editing. Mündler et al. (2024) observe that there is no correlation between an LLM’s ability to fix a bug and its ability to reproduce it, making test reproduction a unique and challenging task that requires targeted training.

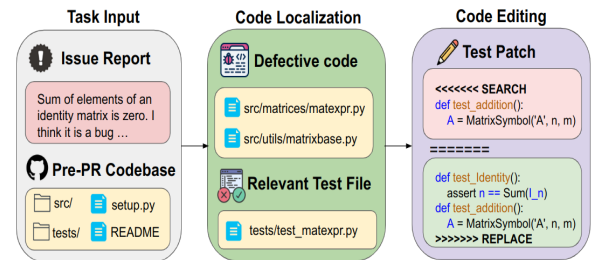


Figure 1: **Our proposed SWE-Tester pipeline.** Given an issue description and the pre-PR repository (which has this issue/bug), we first retrieve defective source code and relevant test file (code localization). Next, we edit the test file to augment it with reproduction tests (code editing).

To address these challenges, we propose **SWE-Tester** – a localized, privacy-preserving framework

* Work done during an internship at Nutanix.

¹<https://github.com/adityasoni9998/SWE-Tester>

for generating issue reproduction tests using fully open-source, open-weights LLMs. As shown in Figure 1, our framework employs a standardized two-step pipeline consisting of code localization followed by code editing. Our analysis reveals that while open-weights models localize code reasonably well, they heavily struggle with editing test files. Since editing is the primary bottleneck, we focus on fine-tuning open-source LLMs to improve their repository-level editing performance. Our contributions are summarized as follows:

1. **Privacy-Preserving Training Dataset:** We leverage raw, open-source GitHub pull requests to curate a high-quality dataset of 41K training instances from 2.6K repositories, allowing local training without exposing proprietary code.
2. **Broad Generalization Across Open Model Families:** We fine-tune five open-source LLMs across varying parameter sizes and architectures: Qwen-2.5-Coder (7B, 14B, and 32B), Llama-3.1-8B, and Gemma-3-12B. Fine-tuning yields absolute gains of up to 10% in success rate and 21% in change coverage on SWT-Bench Verified, outperforming strong closed-source baselines like ZeroShotPlus (Mündler et al., 2024).
3. **Operational Cost and Scaling Analysis:** We analyze the trade-offs of inference-time compute scaling (sampling up to 32 candidate patches). Our results show consistent performance gains without saturation, offering concrete guidance on balancing execution latency, token costs, and safety sandboxing in enterprise CI/CD pipelines.

2 Related Work

Issue Reproduction Benchmarks and Training.

Issue reproduction is substantially more challenging than conventional unit test generation because tests must be derived from a natural language issue description and a buggy repository without access to the issue-fixing patch. SWT-Bench (Mündler et al., 2024) and Defects4J (Just et al., 2014) are widely used benchmarks for evaluating this capability. While SWT-Bench supports both *unit test* and *reproduction script* settings, prior training efforts primarily focus on reproduction scripts or depend heavily on proprietary models. For example, Jain et al. (2025) train testing agents using trajectories generated by closed-source LLMs, while He et al.

(2025) employ reinforcement learning on pipeline-based workflows. In contrast, we train open-source LLMs directly for the more realistic *unit test* setting using developer-written tests mined from historical pull requests.

Reproduction Test Generation Workflows.

Existing approaches typically employ multi-stage pipelines or agentic workflows involving planning, localization, root-cause analysis, and iterative refinement (Ahmed et al., 2025a; Kang et al., 2023; Wang et al., 2024; Mündler et al., 2024; Khatib et al., 2025). Although effective, these systems frequently rely on proprietary LLMs and generate intermediate reasoning steps for which supervised training data is unavailable. We instead adopt a simple two-stage workflow consisting of localization and editing, enabling straightforward supervised fine-tuning from raw pull request data.

Automated Issue Resolution. Issue resolution focuses on modifying source code to fix bugs and is commonly evaluated on SWE-Bench and its variants (Jimenez et al., 2024; Yang et al., 2025a). Recent training approaches either train LLM agents (Pan et al., 2025; Jain et al., 2025; Yang et al., 2025b) or specialized pipeline components (Wei et al., 2025; Xie et al., 2025; He et al., 2025). We leverage the large-scale pull request data collected by these efforts but repurpose it to train models for issue reproduction, a related yet distinct task.

3 Methodology

In this section, we formally define the issue reproduction task in §3.1, describe our workflow in §3.2, explain our data curation and training methodology in §3.3, and finally describe the inference scaffold used to leverage inference-time compute in §3.4.

3.1 Issue Reproduction Task

We formally define the issue reproduction task following Mündler et al. (2024). Given a code repository $\mathcal{R}$, a natural language issue description $\mathcal{I}$, which is a bug report or a feature request in $\mathcal{R}$, and a golden code patch $\mathcal{X}^*$, containing source code changes from the issue-resolving PR. We denote the codebase $\mathcal{R}$ after applying a patch $\mathcal{X}$ as $\mathcal{R} \circ \mathcal{X}$. A test s can either pass (P) or fail (F) when executed on $\mathcal{R}$. A test s fails if its execution triggers an error (for eg: AssertionError, ValueError, etc.), and it passes otherwise. A test s reproduces an issue $\mathcal{I}$ if it fails on the pre-PR codebase $\mathcal{R}$ and passes on the post-PR codebase $\mathcal{R} \circ \mathcal{X}^*$. Such

tests are defined as fail-to-pass tests ($F \rightarrow P$), and $F \rightarrow F$, $P \rightarrow P$ and $P \rightarrow F$ tests are defined similarly. A set of tests T reproduces an issue if: at least one test in T is $F \rightarrow P$ (i.e it correctly exposes the issue), and no test in T fails on $\mathcal{R} \circ \mathcal{X}^*$ (avoiding $F \rightarrow F$ and $P \rightarrow F$ tests with incorrect testing logic or syntax issues). Also, this allows $P \rightarrow P$ regression tests in T . Given the pre-PR codebase $\mathcal{R}$ and the issue description $\mathcal{I}$, the task of the LLM is to generate a test set T that reproduces the issue.

3.2 Workflow for Issue Reproduction

In this section, we describe our two-step workflow for generating issue reproduction tests (Figure 1) using **fully open-source LLMs**. The first step is code localization – retrieving relevant source code and test code from the pre-PR codebase. The second step is code editing – generating a patch to modify the retrieved test code and augmenting it with issue reproduction tests.

3.2.1 Code Localization

For code localization, we retrieve potentially defective source code files and a relevant test file from the pre-PR codebase. The source code files give context on possible root causes and highlight segments that require further testing for issue reproduction. The test file helps the LLM to understand existing testing conventions, identify gaps in testing logic critical for issue reproduction, and reuse relevant utility functions and classes.

Prior pipeline-based approaches follow similar localization steps, with some localizing to more fine-grained locations (e.g., classes or functions) by prompting an LLM (Khatib et al., 2025; Ahmed et al., 2025a). We provide the complete content of the retrieved files following training methods for issue resolution, which find that including the entire file contents improves performance by teaching the model to identify relevant code segments (Xie et al., 2025; Wei et al., 2025). For test code, we follow prior work (Ahmed et al., 2024, 2025a,b), that achieve strong performance on this task, and localize a single test file. While this may miss scenarios like utility code and tests being edited in separate files, it simplifies localization during inference and aids training data filtering (§3.3).

For source code, we adopt the coarse-to-fine retrieval strategy of Xie et al. (2025): BM25 (Robertson et al., 2009) retrieves the top-30 relevant source code files using issue description as the query, and a fine-tuned Qwen2.5-7B (Team, 2024) model then

selects the defective files given skeletons of these 30 files. For test code, we use BM25 to retrieve the top-30 candidate test files with issue being the query, which are then reranked using Qwen3-Embedding-8B (Zhang et al., 2025) model. Since the top-ranked file is not always correct, we use an inference scaffold (§3.4) that samples multiple test patches from the LLM for the top-K test files and reranks them to select the most appropriate patch. Although we can train a retriever model for test files, we find code editing step to be the main bottleneck (§5.1).

3.2.2 Code Editing

The next step is code editing, where the LLM generates a patch to add reproduction tests to the test file given the issue and the localized context. A key design choice is the patch format: while the unified diff format (used in `git diff`) is a standard representation, it is highly sensitive to mis-specifications since LLMs often struggle with precise line numbers (Jimenez et al., 2024; Mündler et al., 2024). Mündler et al. (2024); Ahmed et al. (2025a) use LLM-friendly formats that only allow insertion and replacement of functions, but they do not support editing other code segments like imports, global constants, class attributes, etc. This is particularly problematic for training data curation because edits to the test suite in developer-written PRs are not restricted to function-level edits. To overcome these limitations, we use the Search/Replace format (Xia et al., 2024; Wei et al., 2025), which can represent any arbitrary file edits (example shown in Appendix A.1) and simplifies training data curation §3.3.

3.3 Dataset Curation and Training Methodology

Next, we describe our data curation and training methodology for the code editing step. Prior training methods for issue resolution have scraped raw PR data from GitHub repositories, with some also developing sandboxed execution environments (Pan et al., 2025; Jain et al., 2025). While they target issue resolution, we instead train models to predict the test patch for issue reproduction – a related but distinct task, since an LLM’s ability to fix an issue does not correlate with its ability to reproduce it (§1). During training, the LLM is prompted with the issue description, the pre-PR contents of the gold defective source code and gold test files edited by the issue-resolving PR. Refer

to Appendix A.1 for the exact prompts used in our work.

To curate training data, we leverage PR data curated by SWE-Gym (Pan et al., 2025), SWE-Fixer (Xie et al., 2025), SWE-rebench (Badertdinov et al., 2025), and SWE-Bench Extra (Badertdinov et al., 2024), which contain a total of 140K issue-PR pairs across 5K repositories. However, these datasets do not provide pre-PR file contents of the source code and test files edited by the PR, so we scrape them by cloning the pre-PR codebase for each instance. Since raw PR data is very noisy, we exclude PRs that satisfy any of these conditions: 1) belongs to repositories present in SWT-Bench (our evaluation set), 2) edits non-Python files, 3) edits more than three or zero source code files, 4) does not edit *exactly* one test file, or 5) has empty issue descriptions. Since not every edit made by the test patch adds a reproduction test, filtering criterion 4) helps ensure that the edits are *generally* relevant to the issue. We further discuss our rationale behind filtering criterion in Appendix A.3. After filtering and de-duplication, we obtain **41K** high-quality training instances from **2.6K** repositories. We write custom scripts to translate developer-written test patches from the unified diff format to Search/Replace format. Table 4 shows dataset statistics and repo-level instance frequencies. Similar to our parent datasets, our data also has a long-tail distribution: while **70%** repositories have <5 instances, the most frequent repository (pandas) has 1529 instances.

3.4 Inference Scaffold

Our localization pipeline (§3.2.1) retrieves defective source files using SWE-Fixer 7B Retriever (Xie et al., 2025) and reranks test files with Qwen3-Embedding-8B (Zhang et al., 2025). Since the top-ranked test file may not be correct, we generate $N = 8$ patches with temperature 0.5 for each of the top- $K = 4$ test files, yielding 32 candidates. All candidates use the same source code context from SWE-Fixer, as prior work (Mündler et al., 2024; Ahmed et al., 2025a) shows test localization has greater impact on model performance than source localization.

From these 32 candidates, we select the most appropriate patch through a multi-step filtering and reranking process. First, we discard all patches that are not applicable to the test file. Next, for each applicable patch, we execute the original test file and the modified test file (after applying the

patch) on the pre-PR codebase. We compare the test execution logs of the two runs and discard all patches which introduce errors (e.g. `SyntaxError`, `ImportError`, etc.).

Since a reproduction test set must have at least one test that fails on the pre-PR codebase (§3.1), we only consider all such patches for the final patch set $\mathcal{S}$. We rerank patches in $\mathcal{S}$ using a self-consistency mechanism similar to (He et al., 2025). We assign a score to each patch p in $\mathcal{S}$, computed as: $\text{Score}(p) = \text{Score}_{\text{EM}}(p) + \text{Score}_{\text{Sim}}(p)$. $\text{Score}_{\text{EM}}(p)$ is the standard self-consistency measure, counting how often p occurs in $\mathcal{S}$. $\text{Score}_{\text{Sim}}(p)$ rewards candidates that lie in dense clusters of plausible variants: we compute pairwise similarity between patches with Python’s `difflib.SequenceMatcher`, identify the top- r nearest neighbors for each candidate, and average their similarity scores. Following He et al. (2025), we set $r = |\mathcal{S}|/2$. The patch with the highest combined score is selected as the final answer. We describe potentially better reranking methods in Appendix A.5

4 Experimental Setup

We perform supervised fine-tuning of Qwen2.5-Coder Instruct (7B, 14B, 32B) (Hui et al., 2024), Llama-3.1-8B Instruct (Dubey et al., 2024), and Gemma-3-12B Instruct (Team et al., 2025). To fit hardware constraints and preserve high training efficiency, we filter out instances exceeding a 16K token limit (using the Qwen2.5-Coder-7B tokenizer), resulting in 23K active training instances. We train all models for 2 epochs with a batch size of 32, a cosine learning rate scheduler, and the AdamW optimizer. The learning rate is set to $1e^{-5}$ for Gemma-3-12B and $3e^{-5}$ for all other models. Training is performed on 2 to 8 NVIDIA A100 GPUs using the torchtune or Llama-Factory libraries.

We evaluate performance on the SWT-Bench Verified dataset (Mündler et al., 2024) (433 instances) using three core metrics: Applicability ($\mathcal{W}$), Success Rate ($\mathcal{S}$), and Change Coverage ($\Delta\mathcal{C}$). Applicability measures the percentage of instances where generated patches apply cleanly. Success Rate is the percentage of instances where the generated tests reproduce the bug ($F \rightarrow P$ on the codebase). Change Coverage represents the fraction of gold patch lines executed by the new tests.

Table 1: Comparing applicability ($\mathcal{W}$), change coverage ($\Delta\mathcal{C}$), and success rate ($\mathcal{S}$) of base and fine-tuned LLMs on SWT-Bench Verified (Mündler et al., 2024) using our inference scaffold (§3.4).

Model	Applicability (% , $\uparrow$)			Change Coverage (% , $\uparrow$)			Success Rate (% , $\uparrow$)		
	base	SFT	Δ	base	SFT	Δ	base	SFT	Δ
Llama-3.1-8B-Instruct	18.01	84.76	+66.75	1.42	22.28	+20.86	0.23	6.24	+6.01
Gemma-3-12B-Instruct	85.91	90.99	+5.08	5.95	24.12	+18.17	0.92	8.08	+7.16
Qwen2.5-Coder-7B-Instruct	40.42	84.99	+44.57	3.23	21.30	+18.07	1.15	8.31	+7.16
Qwen2.5-Coder-14B-Instruct	30.95	84.99	+54.04	2.51	23.54	+21.03	1.39	11.79	+10.40
Qwen2.5-Coder-32B-Instruct	84.76	84.76	0.00	19.14	22.73	+3.59	9.93	14.09	+4.16

5 Results

Table 1 presents our main evaluation results. We outline the key engineering takeaways below:

Local SFT consistently improves bug reproduction tests: Fine-tuning on our curated dataset significantly enhances success rates and change coverage across all five open LLMs. Smaller base models (≤ 14 B) struggle with base test generation, achieving $\leq 1.39\%$ success rate. However, SFT yields massive improvements of up to $+21.03\%$ in change coverage and $+10.40\%$ in success rate. While Qwen2.5-Coder-32B shows more modest relative SFT gains ($+4.16\%$ success rate), it achieves the highest overall success rate of 14.09% . Crucially, these results are achieved using fully open-weights models, offering a privacy-preserving and cost-effective local deployment alternative to closed APIs.

Model scale and architecture govern downstream quality: Fine-tuned performance generally scales with parameter size. Within the Qwen2.5-Coder family, success rates scale predictably from 7B (8.31%) to 32B (14.09%). However, base model choice plays an equally vital role; despite their comparable scales, Llama-3.1-8B-Instruct underperforms Qwen2.5-Coder-7B-Instruct by 2.07% in SFT success rate, and Gemma-3-12B-Instruct underperforms Qwen2.5-Coder-14B-Instruct by 3.71% .

Fine-tuning resolves syntactic patch formatting issues: A key barrier to automated test execution is generating syntactically valid patches. Base models (excluding 32B and Gemma-3) fail to generate cleanly applicable patches. SFT resolves this formatting bottleneck, boosting applicability to $\geq 84.76\%$ across all fine-tuned variants. Interestingly, Gemma-3 achieves high applicability (90.99% SFT) but a lower success rate (8.08%), highlighting that generating well-formed patches is a necessary but insufficient condition for successful bug reproduction.

While the focus of our work is to investigate how we can improve the performance of open-source LLMs for this task, we include comparisons with prior methods in Appendix A.8.

5.1 Retrieval Performance for Code Localization

These results evaluate our retrieval methods and demonstrate that code editing (rather than localization) remains the primary bottleneck for bug reproduction.

Source Code Localization Table 2 evaluates source file localization on SWT-Bench Verified. The fine-tuned SWE-Fixer retriever (Xie et al., 2025) achieves high precision (58.7%) and recall (58.7%). While BM25 Top-30 achieves a higher recall (87.7%), its extremely low precision (3.4%) introduces excessive context noise, justifying our coarse-to-fine localization strategy.

Table 2: Source code localization performance on SWT-Bench Verified.

Method	Precision (%)	Recall (%)
BM25 Top-1	39.6	43.6
BM25 Top-4	16.9	59.7
BM25 Top-30	3.4	87.7
SWE-Fixer Retrieval	58.7	58.7

Test Localization Table 3 evaluates single test file retrieval. Reranking BM25 candidates with Qwen3-Embedding (Zhang et al., 2025) substantially increases recall for small K . Reranking with the 0.6B model achieves up to 73.0% recall, performing almost identically to the 8B model while significantly lowering inference latency and deployment costs.

Table 3: Test file localization comparing BM25 with BM25 Top-30 + Qwen3-Embedding reranking (all metrics in %).

Top- K	BM25		BM25 Top-30 + Reranking			
			0.6B		8B	
	Prec.	Rec.	Prec.	Rec.	Prec.	Rec.
1	30.0	27.2	51.5	46.6	53.3	47.9
2	22.7	39.7	32.6	57.4	35.0	60.7
4	15.6	53.5	19.2	65.4	19.7	67.0
8	9.8	64.7	10.9	70.9	10.9	71.9
16	5.9	74.1	5.9	73.0	5.9	73.1

6 Analysis

In this section, we analyze the operational factors that influence the performance and practical deployment of open-weights models for bug reproduction. We evaluate the impact of inference-time compute scaling in §6.1 and analyze repository-level performance variations in §6.2. Additional evaluations of offline training data scaling, parameter-efficient fine-tuning (LoRA), and edit-only "oracle" baseline comparisons are detailed in Appendices A.6, A.7, and A.9.

6.1 Effect of Scaling Inference-Time Compute

We evaluate how model performance scales with increased test-time compute. We hold the number of retrieved test files constant ($K = 4$) and vary the number of sampled patches per file ($N \in \{1, 2, 4, 8\}$), resulting in a budget of 4, 8, 16, and 32 candidate patches per issue. The final reproduction patch is selected using our execution-backed filtering and self-consistency scaffold (§3.4).

As shown in Figure 2, we observe consistent, monotonic performance gains across the Qwen2.5-Coder family (7B, 14B, and 32B). Notably, neither success rate ($\mathcal{S}$) nor change coverage ($\Delta\mathcal{C}$) shows signs of saturation up to 32 candidate samples. This trend demonstrates that test-time compute scaling is a highly viable strategy for local enterprise deployments: organizations can directly trade off additional local GPU compute cycles for significantly higher test generation quality, all while maintaining complete data privacy.

6.2 Analyzing Performance Across Repositories

Using the inference setup from §5, we analyze the distribution of successfully reproduced bugs across different software repositories to identify domain-specific model behavior. Detailed repository-level breakdown metrics are presented in Table 8 in Appendix A.10.

We find that repository characteristics heavily influence model performance. All fine-tuned models demonstrate high proficiency on lightweight web frameworks such as flask (achieving up to 100% success rate) but struggle on complex, highly integrated documentation engines like sphinx (under 4% success rate) and plotting libraries like seaborn (0% success rate). This performance gap is primarily driven by: (1) environment and dependency complexity, where graphics-rendering or special-

ized CLI dependencies are harder to initialize in sandbox environments, and (2) long-context dependencies, where complex systems require larger context horizons to localize relevant helper functions. Interestingly, model scaling is not strictly monotonic; the 7B model outperforms the larger 14B and 32B variants on data-focused repositories such as xarray (26.67% vs. 6.67%), indicating that localized file conventions are occasionally captured better by smaller, highly focused SFT checkpoints.

7 Limitations

While our work demonstrates strong results for training LLMs for issue reproduction, it has several limitations.

First, we evaluate only Python repositories due to the lack of large-scale open-source benchmarks for other languages (e.g., only Defects4J (Just et al., 2014) for Java and SWT-Bench (Mündler et al., 2024) for Python).

Second, our training data is constructed using heuristic filtering under computational constraints, and we do not extensively study alternative filtering strategies. We release our data curation pipeline to support future exploration.

Third, we do not explicitly train models for test localization, instead relying on off-the-shelf LLMs, and similarly do not optimize our inference scaffold for best-in-class reranking or patch filtering, as our focus is on studying scalable training for issue reproduction rather than system optimization.

Fourth, we do not verify whether all training patches truly reproduce the underlying issue due to the lack of fully sandboxed execution environments in a large portion of the dataset (notably SWE-Fixer-derived data (Xie et al., 2025)). Building such environments is expensive or requires proprietary agent systems.

Finally, we assume single-test-file edits, which simplifies the task but limits applicability to multi-file test refactoring scenarios. Extending to multi-file settings and improving performance via stronger inference-time reasoning or reinforcement learning remains future work.

References

Toufique Ahmed, Jatin Ganhotra, Rangeet Pan, Avraham Shinnar, Saurabh Sinha, and Martin Hirzel. 2025a. *Otter: Generating tests from issues to validate SWE patches*. In *Forty-second International Conference on Machine Learning*.

- Toufique Ahmed, Jatin Ganhotra, Avraham Shinnar, and Martin Hirzel. 2025b. [Execution-feedback driven test generation from swe issues](#). *Preprint*, arXiv:2508.06365.
- Toufique Ahmed, Martin Hirzel, Rangeet Pan, Avraham Shinnar, and Saurabh Sinha. 2024. Tdd-bench verified: Can llms generate tests for issues before they get resolved? *arXiv preprint arXiv:2412.02883*.
- Ibragim Badertdinov, Alexander Golubev, Maksim Nekrashevich, Anton Shevtsov, Simon Karasik, Andrei Andriushchenko, Maria Trofimova, Daria Litvintseva, and Boris Yangel. 2025. [Swe-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents](#). *Preprint*, arXiv:2505.20411.
- Ibragim Badertdinov, Maria Trofimova, Yuri Anapolskiy, Sergey Abramov, Karina Zainullina, Alexander Golubev, Sergey Polezhaev, Daria Litvintseva, Simon Karasik, Philipp Fisin, Sergey Skvortsov, Maxim Nekrashevich, Anton Shevtsov, and Boris Yangel. 2024. [Scaling data collection for training software engineering agents](#). *Nebius blog*.
- Kent Beck. 2022. *Test driven development: By example*. Addison-Wesley Professional.
- Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, and 1 others. 2024. The llama 3 herd of models. *arXiv e-prints*, pages arXiv–2407.
- Zhenyu He, Qingping Yang, Wei Sheng, Xiaojian Zhong, Kechi Zhang, Chenxin An, Wenlei Shi, Tianle Cai, Di He, Jiaze Chen, Jingjing Xu, and Mingxuan Wang. 2025. [Swe-swiss: A multi-task fine-tuning and rl recipe for high-performance issue resolution](#).
- Binyuan Hui, Jian Yang, Zeyu Cui, Jiaxi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Keming Lu, and 1 others. 2024. Qwen2. 5-coder technical report. *arXiv preprint arXiv:2409.12186*.
- Naman Jain, Jaskirat Singh, Manish Shetty, Tianjun Zhang, Liang Zheng, Koushik Sen, and Ion Stoica. 2025. [R2e-gym: Procedural environment generation and hybrid verifiers for scaling open-weights SWE agents](#). In *Second Conference on Language Modeling*.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. 2024. [SWE-bench: Can language models resolve real-world github issues?](#) In *The Twelfth International Conference on Learning Representations*.
- René Just, Darioush Jalali, and Michael D Ernst. 2014. Defects4j: A database of existing faults to enable controlled testing studies for java programs. In *Proceedings of the 2014 international symposium on software testing and analysis*, pages 437–440.
- Sungmin Kang, Juyeon Yoon, and Shin Yoo. 2023. Large language models are few-shot testers: Exploring llm-based general bug reproduction. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 2312–2323. IEEE.
- Lara Khatib, Noble Saji Mathews, and Meiyappan Nagappan. 2025. Assertflip: Reproducing bugs via inversion of llm-generated passing tests. *arXiv preprint arXiv:2507.17542*.
- Ilya Loshchilov and Frank Hutter. 2019. [Decoupled weight decay regularization](#). *Preprint*, arXiv:1711.05101.
- Niels Mündler, Mark Niklas Mueller, Jingxuan He, and Martin Vechev. 2024. [SWT-bench: Testing and validating real-world bug-fixes with code agents](#). In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Jiayi Pan, Xingyao Wang, Graham Neubig, Navdeep Jaitly, Heng Ji, Alane Suhr, and Yizhe Zhang. 2025. [Training software engineering agents and verifiers with swe-gym](#). In *Proceedings of the 42nd International Conference on Machine Learning (ICML 2025)*. ArXiv:2412.21139, accepted at ICML 2025.
- Stephen Robertson, Hugo Zaragoza, and 1 others. 2009. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends® in Information Retrieval*, 3(4):333–389.
- Philipp Straubinger and Gordon Fraser. 2023. A survey on what developers think about testing. In *2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE)*, pages 80–90. IEEE.
- Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivière, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot, Ivo Penchev, and 197 others. 2025. [Gemma 3 technical report](#). *Preprint*, arXiv:2503.19786.
- Qwen Team. 2024. [Qwen2.5: A party of foundation models](#).
- Xinchen Wang, Pengfei Gao, Xiangxin Meng, Chao Peng, Ruida Hu, Yun Lin, and Cuiyun Gao. 2024. [Aegis: An agent-based framework for general bug reproduction from issue descriptions](#). *Preprint*, arXiv:2411.18015.
- Yuxiang Wei, Olivier Duchenne, Jade Copet, Quentin Carbonneaux, Lingming Zhang, Daniel Fried, Gabriel Synnaeve, Rishabh Singh, and Sida I. Wang. 2025. Swe-rl: Advancing llm reasoning via reinforcement learning on open software evolution. *arXiv preprint arXiv:2502.18449*.
- Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489*.

Chengxing Xie, Bowen Li, Chang Gao, He Du, Wai Lam, Difan Zou, and Kai Chen. 2025. Swe-fixer: Training open-source llms for effective and efficient github issue resolution. *arXiv preprint arXiv:2501.05040*.

John Yang, Carlos E Jimenez, Alex L Zhang, Kilian Lieret, Joyce Yang, Xindi Wu, Ori Press, Niklas Muennighoff, Gabriel Synnaeve, Karthik R Narasimhan, Diyi Yang, Sida Wang, and Ofir Press. 2025a. *Swe-bench multimodal: Do ai systems generalize to visual software domains?* In *The Thirteenth International Conference on Learning Representations*.

John Yang, Kilian Lieret, Carlos E. Jimenez, Alexander Wettig, Kabir Khandpur, Yanzhe Zhang, Binyuan Hui, Ofir Press, Ludwig Schmidt, and Diyi Yang. 2025b. *Swe-smith: Scaling data for software engineering agents*. *Preprint*, arXiv:2504.21798.

Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, and 1 others. 2025. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*.

A Appendix

A.1 Prompt Template for Issue Reproduction

We provide the prompt used for both training and inference. Training uses gold source and test files, while inference uses outputs from the localization pipeline (§3.2.1).

Issue Reproduction Prompt

System Prompt:

Generate fail-to-pass test edits for a GitHub issue. Do not modify source code. Output must be enclosed in a `<solution>` block and follow SEARCH/REPLACE format.

User Prompt:

Given a GitHub issue, source files, and a test file, modify the test file to reproduce the issue.

Output Format:

- File path
- <<<<<<< SEARCH
- Original code
- =====
- Modified code
- >>>>>>> REPLACE

A.2 Test File Reranking Prompt

We use Qwen3-Embedding (Zhang et al., 2025) to rerank BM25-retrieved test files.

Reranking Prompt

Instruction: Given a GitHub issue and a test file, determine whether it contains sufficient context (classes, functions, boilerplate) to support adding fail-to-pass tests that reproduce the issue.

Query: {Issue description}

A.3 Dataset Filtering Rationale

We apply filtering to improve data quality under compute constraints:

1. Remove SWT-Bench repositories to avoid leakage.
2. Exclude non-Python edits (e.g., docs, configs).
3. Limit to ≤ 3 source files to reduce complexity.
4. Keep PRs with exactly one test file, following prior work (Xie et al., 2025).
5. Remove empty issue descriptions due to missing supervision signal.

A.4 Training Details

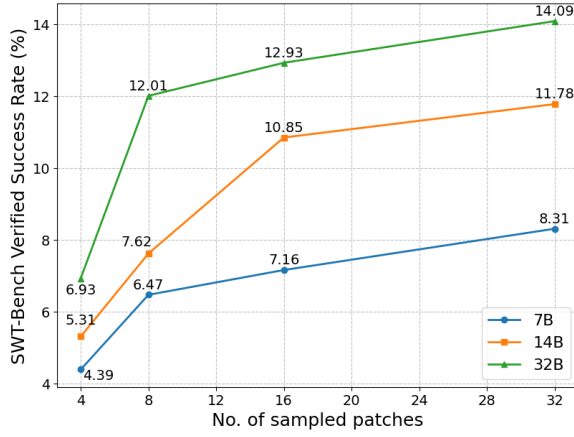
We train for 2 epochs using batch size 32, AdamW (Loshchilov and Hutter, 2019), cosine LR scheduling, and gradient clipping (norm 1.0). Learning rates are $1e^{-5}$ for Gemma-3-12B and $3e^{-5}$ for others. Training uses torchtune or Llama-Factory (Gemma). Experiments run on 2–8× A100 GPUs.

A.5 Reranking Strategy

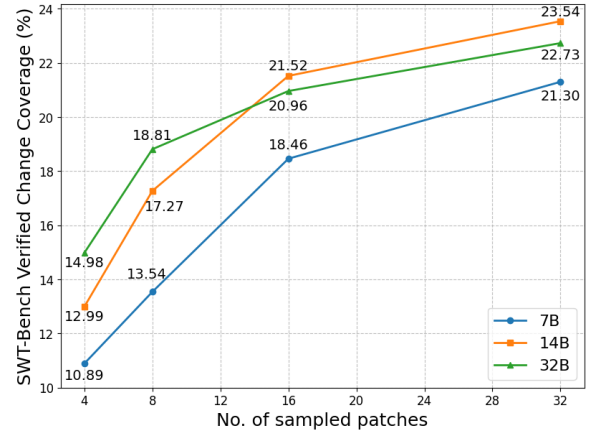
We observe that change coverage (ΔC) is 2–3× higher for successful patches than failed ones, making it a stronger ranking signal than self-consistency. In practice, this can be estimated using candidate source patches from systems like Agentless (Xia et al., 2024).

A.6 Oracle Retrieval Evaluation

We evaluate edit-only capability using oracle retrieval (gold source and test files provided). We sample with temperature 0 and retry up to 10 times with incremental temperature increases.

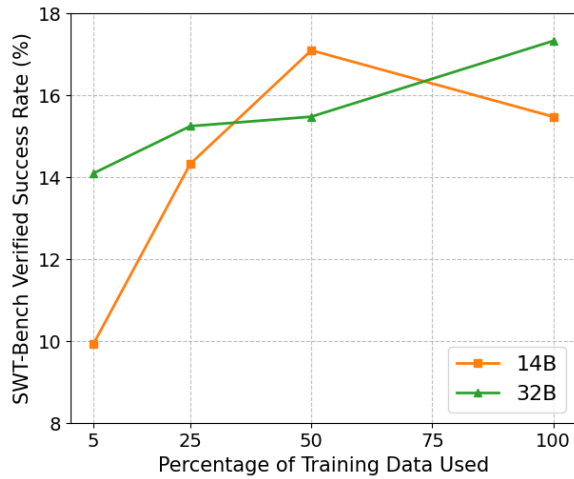


(a) Success Rate ($\mathcal{S}$)

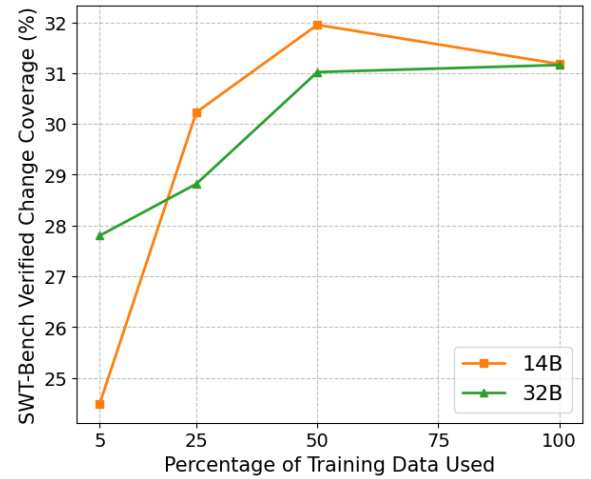


(b) Change Coverage ($\Delta\mathcal{C}$)

Figure 2: Inference-time scaling on SWT-Bench Verified. Increasing sampled candidate patches consistently improves performance without saturation.



(a) Success Rate



(b) Change Coverage

Figure 3: Scaling trends with increasing training data.

Category	Metric	Dataset
Size	# Instances	41K
	# Repos	2.6K
Issue Text	# words	175.7
Gold code patch	# Files edited	1.4
	# Lines edited	37.2
Gold test patch	# Files edited	1
	# Lines edited	47.9

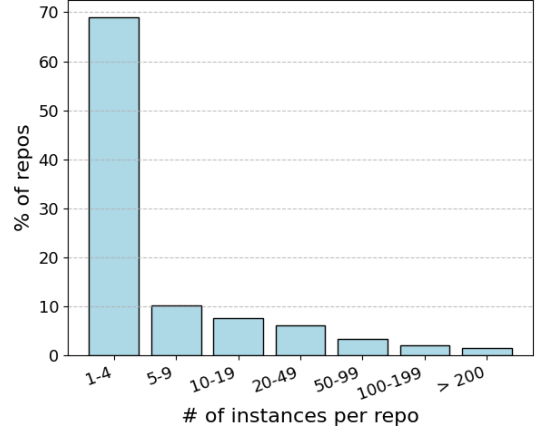


Table 4: Dataset statistics for our training dataset (left) and histogram of repo-level instance frequencies (right). Except for size, we report the instance-level averages for all metrics in the table.

Model	Applicability		Coverage		Success	
	base	SFT	base	SFT	base	SFT
Qwen2.5-7B	40.42	84.99	3.23	21.30	1.15	8.31
Qwen2.5-14B	30.95	84.99	2.51	23.54	1.39	11.79
Qwen2.5-32B	84.76	84.76	19.14	22.73	9.93	14.09

Table 5: Oracle retrieval evaluation on SWT-Bench Verified.

A.7 Training Data Scaling

We train on 5%, 25%, 50%, and 100% of 23K instances under oracle retrieval.

Performance improves consistently with data size. The 14B model saturates around 50%, while 32B continues improving, showing higher capacity. Even 5% data yields strong gains.

A.8 Prior Benchmark Comparison

We compare against closed-source and open models on SWT-Bench Verified.

Method	Model	Success
ZeroShotPlus	GPT-4o	14.30
LIBRO	GPT-4o	17.80
SWE-Tester 7B	Qwen2.5-7B	8.31
SWE-Tester 14B	Qwen2.5-14B	11.79
SWE-Tester 32B	Qwen2.5-32B	14.09

Table 6: Comparison on SWT-Bench Verified.

Our 32B model is competitive with GPT-4o-based pipelines despite being fully open-weight.

A.9 PEFT Evaluation

We evaluate LoRA ($r=8$, $\alpha = 16$) on Qwen2.5-Coder models.

PEFT yields strong gains, showing full fine-tuning is not required.

Model	Applicability		Coverage		Success	
	base	SFT	base	SFT	base	SFT
LoRA-7B	10.62	69.28	2.03	25.74	0.69	10.39
LoRA-32B	73.74	68.12	24.55	29.16	11.32	16.86

Table 7: LoRA fine-tuning under oracle retrieval.

A.10 Repository Breakdown

We report per-repository success rates for Qwen2.5-Coder models.

Repository	7B	14B	32B
astropy	5.88	23.53	23.53
django	7.87	10.19	11.11
matplotlib	0.00	9.38	18.75
seaborn	0.00	0.00	0.00
flask	0.00	100.00	100.00
requests	25.00	0.00	0.00
xarray	26.67	6.67	6.67
pylint	0.00	16.67	33.33
pytest	13.33	13.33	26.67
scikit-learn	16.67	25.00	25.00
sphinx	3.57	3.57	3.57
sympy	8.22	13.70	16.44

Table 8: Repository-wise performance breakdown.

A.11 Dataset License

All data is derived from publicly available GitHub repositories under permissive licenses (MIT/Apache/BSD). We respect original licensing constraints and release curation scripts for compliance.